

Data Engineering Curriculum

Day 35: Delta Lake Deep Dive & Lakehouse Architecture

Day 35: Delta Lake Core Concepts

Building on Day 34's PySpark transformations, Day 35 focuses on **Delta Lake**. Delta Lake is an open-source storage layer that brings reliability, performance, and lifecycle management to data lakes.

- **ACID Transactions:** Ensures data integrity even if a Spark job fails mid-write. Multiple writers can simultaneously modify a dataset without corrupting it.
- **Time Travel (Data Versioning):** Delta Lake maintains a transaction log (the `_delta_log` folder). This allows you to query earlier versions of the table, roll back accidental deletes, or reproduce machine learning experiments.
- **Schema Enforcement & Evolution:** Automatically prevents bad data (e.g., an extra column) from being written to your table, but allows you to explicitly evolve the schema when requirements change.

Advanced Delta Operations

- **UPSERTS (Merge):** Unlike standard Parquet files which are append-only, Delta Lake allows you to use `MERGE INTO` to update existing records and insert new ones at the same time.
- **OPTIMIZE & Z-Ordering:** Solves the "small file problem" by compacting small files into larger, more efficient ones. Z-Ordering co-locates related data in the same files to dramatically speed up queries.
- **VACUUM:** Removes old data files that are no longer referenced by the transaction log, saving storage costs.

End-to-End Pipeline: ADF Orchestrating Delta Upserts

Problem Statement: Every day, a new batch of e-commerce orders (inserts and updates) arrives as `daily_sales.csv` in a source Azure Data Lake container. Use Azure Data Factory to trigger a Databricks notebook. The PySpark script must read this CSV and perform an **UPSERT (MERGE)** into an existing Delta Lake table. Finally, run maintenance tasks to compact the files.

Part 1: The Orchestration Flow (ADF)

1. **Event-Based Trigger:** In ADF, set up a Storage Event Trigger that listens for the arrival of `daily_sales.csv`.

2. **Databricks Activity:** Connect the trigger to a Databricks Notebook Activity using an interactive or job cluster.
3. **Parameter Passing:** Pass the filename or date as a base parameter from ADF to Databricks so the script knows exactly which file to process.

Part 2: The PySpark Delta Logic

Below is the PySpark code for your Databricks notebook to handle the Upsert and optimization tasks.

```

from delta.tables import DeltaTable
from pyspark.sql.functions import col

# 1. Define ADLS Paths
source_csv_path = "abfss://landing@dfs.core.windows.net/daily_sales.csv"
delta_table_path = "abfss://gold@dfs.core.windows.net/delta/sales_master"

# 2. Read the daily incremental data
df_daily_updates = spark.read \
    .format("csv") \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .load(source_csv_path)

# 3. Perform the UPSERT (Merge) operation
if DeltaTable.isDeltaTable(spark, delta_table_path):
    # Table exists, perform MERGE
    deltaTable = DeltaTable.forPath(spark, delta_table_path)

    deltaTable.alias("target").merge(
        df_daily_updates.alias("source"),
        "target.order_id = source.order_id" # The unique key to match on
    ).whenMatchedUpdateAll(
        # If order_id exists, update all columns
    ).whenNotMatchedInsertAll(
        # If order_id is new, insert the row
    ).execute()
    print("Upsert completed successfully.")

else:
    # Table does not exist yet (First Run), create it
    df_daily_updates.write \
        .format("delta") \
        .mode("overwrite") \
        .save(delta_table_path)
    print("Initial Delta table created.")

# 4. Maintenance: Optimize and Vacuum
# Compact small files and organize by customer_id for faster querying
spark.sql(f"OPTIMIZE delta.`{delta_table_path}` ZORDER BY (customer_id)")

# Clean up physical files older than 7 days (default retention)
spark.sql(f"VACUUM delta.`{delta_table_path}`")

```